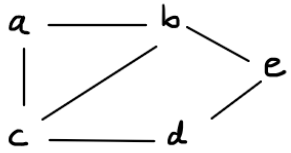
1. (4 Punkte) Gegeben ist der abgebildete ungerichtete Graph.



   (a) (1 Punkt) Welchen Grad hat der Knoten b?

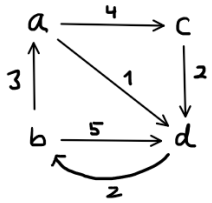
   (b) (1 Punkt) Welche Nachbarn hat der Knoten d?

   (c) (1 Punkt) Gib einen Weg von a nach e an, der über d führt.

   (d) (1 Punkt) Gib einen Kreis an, der vier Knoten enthält.

Lösung: a) 3
b) c und e
c) a-c-d-e
d) b-c-d-e-b

2. (4 Punkte) Gegeben ist der abgebildete gerichtete Graph.



   (a) (1 Punkt) Wie groß sind Eingangsgrad und Ausgangsgrad von d?

   (b) (1 Punkt) Welche Knoten sind Nachfolger von b?

   (c) (1 Punkt) Gib einen Kreis an. Welche Kosten hat er?

   (d) (1 Punkt) Ist der Graph zyklisch oder azyklisch? Begründe.

Lösung: a) Eingangsgrad 3, Ausgangsgrad 1
b) a und d
c) a-d-b-a mit den Kosten $1 + 2 + 3 = 6$
d) Zyklisch, denn er enthält einen Kreis.

3. (3 Punkte) Die Situationen sollen als Graph modelliert werden. Was sind die Knoten, was sind die Kanten? Ist der Graph gerichtet? Ist er gewichtet?

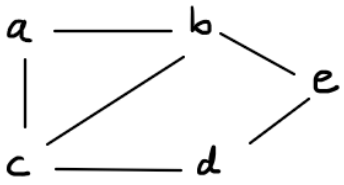
   (a) (1 Punkt) Freundschaften in einem sozialen Netzwerk

   (b) (1 Punkt) ein Straßennetz mit Einbahnstraßen, gesucht sind kurze Strecken

   (c) (1 Punkt) die Arbeitsschritte beim Hausbau, wobei manche Schritte vor anderen erledigt sein müssen

Lösung: a) Knoten: Personen, Kanten: befreundet. Ungerichtet und ungewichtet.
b) Knoten: Kreuzungen, Kanten: Straßen. Gerichtet und gewichtet mit der Länge.
c) Knoten: Arbeitsschritte, Kanten: a muss vor b geschehen. Gerichtet und ungewichtet.

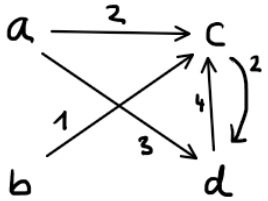
4. (3 Punkte) Gib für den abgebildeten Graphen die Implementierung in Python als Adjazenzmatrix an. Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet.



Lösung:

```
G = [[0,1,1,0,0],
     [1,0,1,0,1],
     [1,1,0,1,0],
     [0,0,1,0,1],
     [0,1,0,1,0]]
```

5. (4 Punkte) Gib für den abgebildeten Graphen die Implementierung in Python als Adjazenzmatrix an. Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet.



Lösung:

```
inf = float('inf')
G = [[0,inf,2,3],
     [inf,0,1,inf],
     [inf,inf,0,2],
     [inf,inf,4,0]]
```

6. (3 Punkte) In der Variablen G sei ein ungewichteter Graph als Adjazenzmatrix implementiert. Die Knoten a,b,c... sind auf die Indizes 0,1,2... abgebildet. Schreibe eine Anweisung, die in der Variablen `nb` eine Liste mit allen Buchstaben speichert, die Nachbarn von b sind. Hinweis: die Funktion `chr(y+97)` ordnet den Zahlen y=0,1,2,... die Buchstaben a,b,c, ... zu.

Lösung:

```
nb = [chr(y+97) for y in range(len(G)) if G[1][y] == 1]
```

7. (3 Punkte) In der Variablen G sei ein gewichteter Graph als Adjazenzmatrix implementiert. Die Knoten a,b,c... sind auf die Indizes 0,1,2... abgebildet. Schreibe eine Anweisung, die in der Variablen `nb` eine Liste mit allen Buchstaben speichert, die Nachbarn von a sind. Hinweis: die Funktion `chr(y+97)` ordnet den Zahlen y=0,1,2,... die Buchstaben a,b,c, ... zu.

Lösung:

```
nb = [chr(y+97) for y in range(len(G)) if y != 0 and G[0][y] < inf]
```

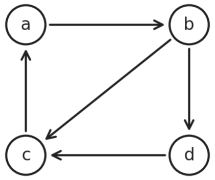
8. (3 Punkte) Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet.

```
G = [[0,1,0,0],
     [0,0,1,1],
     [1,0,0,0],
     [0,0,1,0]]
```

   (a) (2 Punkte) Zeichne den Graphen.

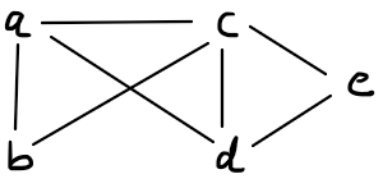
   (b) (1 Punkt) Woran erkennt man an der Matrix, dass der Graph gerichtet ist?

Lösung:



b) Die Matrix ist nicht symmetrisch, zum Beispiel ist G[0][1] gleich 1, aber G[1][0] gleich 0.

9. (4 Punkte) a. Gib für den abgebildeten Graphen die im Unterricht im Abschnitt 'Adjazenzlisten' vorgeschlagene Implementierung in Python an.
   b. Mit welcher Schleife können wir alle Nachbarn des Knoten 'c' durchlaufen?
   c. Welcher boolesche Ausdruck überprüft, ob es eine Kante von 'a' nach 'd' gibt?
   Die Antworten für b. und c. sollen auch für andere Ausprägungen des Graphen gelten.



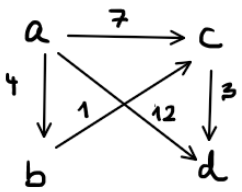
Lösung:

```
a. G = {
'a': set('bcd'),
'b': set('ca'),
'c': set('abde'),
'd': set('ace'),
'e': set('cd')
}

b. for v in G['c']:
     # do something with v

c. 'd' in G['a']
```

10. (4 Punkte) Gib für den abgebildeten Graphen die Implementierung in Python als ein dictionary von dictionaries an.



Lösung:

```
G = {
 'a': {'c':7, 'b':4, 'd':12},
 'b': {'c':1},
 'c': {'d':3},
 'd': {}
}
```

11. (3 Punkte) Der Graph G ist durch Adjazenzlisten implementiert. Gib die Adjazenzmatrix an. Die Knoten a,b,c,.. sind auf die Indizes 0,1,2,... abgebildet.

```
G = {'a': set('bd'),
     'b': set('c'),
     'c': set(),
     'd': set('ac')}
```

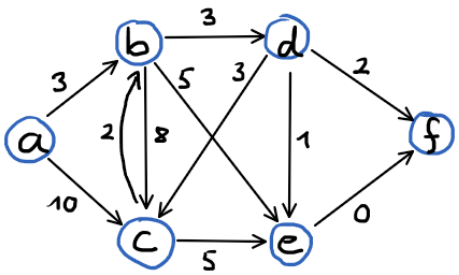
Lösung:

```
M = [[0,1,0,1],
     [0,0,1,0],
     [0,0,0,0],
     [1,0,1,0]]
```

12. (3 Punkte) Ein Graph hat 1000 Knoten und 1500 Kanten.
    (a) (1 Punkt) Wie viele Einträge hat seine Adjazenzmatrix?
    (b) (1 Punkt) Welche Implementation braucht hier weniger Platz? Begründe.
    (c) (1 Punkt) Welchen Vorteil hat die Adjazenzmatrix trotzdem?

Lösung: a) 1000000
b) Die Adjazenzlisten, denn sie speichern nur die 1500 vorhandenen Kanten. Der Graph ist dünn besetzt.
c) Ob es eine Kante von i nach j gibt, lässt sich in konstanter Zeit nachsehen.

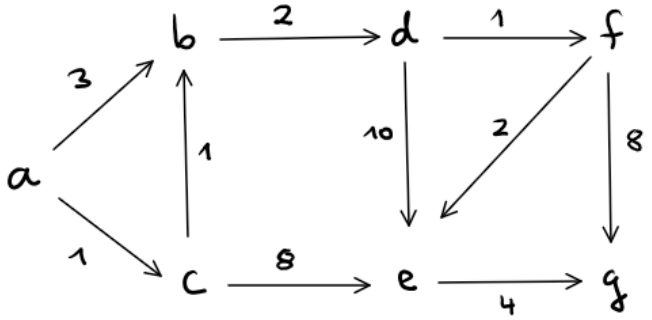
13. (6 Punkte) Ermittle mit dem Algorithmus von Dijkstra die kürzesten Wege von Knoten a zu allen anderen Knoten.
    a. Notiere die Reihenfolge der endgültig markierten Knoten.
    b. Notiere für jeden Knoten die Reihenfolge der Werte, mit denen er markiert wird.



Lösung:

```
Reihenfolge der endgültigen Markierungen: a b d e f c
a : 0
b : inf 3
c : inf 10 9
d : inf 6
e : inf 8 7
f : inf 8 7
```

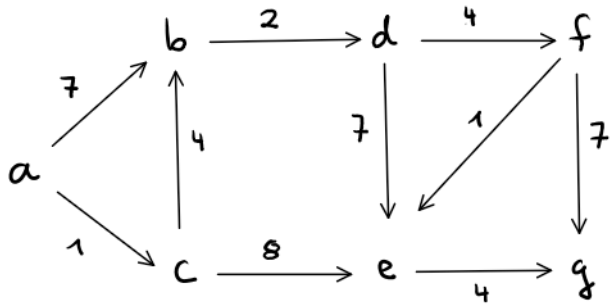
14. (4 Punkte) Ermittle mit dem Algorithmus von Dijkstra die kürzesten Wege von Knoten a zu allen anderen Knoten. Notiere die Reihenfolge der endgültig markierten Knoten. Notiere für jeden Knoten die Reihenfolge der Werte, mit denen er markiert wird.



Lösung:

```
Reihenfolge der endgültigen Markierungen: a c b d f e g
a : 0
b : inf 3 2
c : inf 1
d : inf 4
e : inf 9 7
f : inf 5
g : inf 13 11
```

15. (4 Punkte) Ermittle mit dem Algorithmus von Dijkstra die kürzesten Wege von Knoten a zu allen anderen Knoten. Notiere die Reihenfolge der endgültig markierten Knoten. Notiere für jeden Knoten die Reihenfolge der Werte, mit denen er markiert wird.



Lösung:

```
Reihenfolge der endgültigen Markierungen: a c b d e f g
a : 0
b : inf 7 5
c : inf 1
d : inf 7
e : inf 9
f : inf 11
g : inf 13
```

16. (3 Punkte) Der Algorithmus von Dijkstra wurde für den Startknoten a ausgeführt. Am Ende stehen in den dictionaries dist und prev die folgenden Werte.

```
dist = {'a':0, 'b':3, 'c':9, 'd':6, 'e':7, 'f':7}
prev = {'a':None, 'b':'a', 'c':'d', 'd':'b', 'e':'d', 'f':'e'}
```

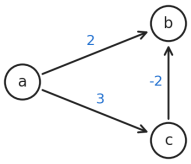
(a) (1 Punkt) Wie lang ist der kürzeste Weg von a nach c?

(b) (1 Punkt) Gib den kürzesten Weg von a nach f an.

(c) (1 Punkt) Welcher Knoten liegt auf dem kürzesten Weg von a nach c unmittelbar vor c?

Lösung: a) 9
b) a-b-d-e-f
c) d, denn prev['c'] ist d.

17. (3 Punkte) Gegeben ist ein Graph mit einem negativen Kantengewicht.



(a) (1 Punkt) In welcher Reihenfolge werden die Knoten endgültig markiert, wenn der Algorithmus von Dijkstra bei a startet?

(b) (1 Punkt) Welchen Wert hat dist(b) am Ende, welche Länge hat der kürzeste Weg von a nach b wirklich?

(c) (1 Punkt) Warum liefert der Algorithmus hier ein falsches Ergebnis?

Lösung: a) a, b, c
b) dist(b) ist 2, der kürzeste Weg a-c-b hat die Länge 1.
c) b wird endgültig, weil er der billigste vorläufige Knoten ist. Das ist nur richtig, wenn kein Umweg billiger sein kann. Die negative Kante macht den späteren Umweg billiger.

18. (4 Punkte) Erkläre in ein bis zwei Sätzen.

(a) (1 Punkt) Was ist der Grad eines Knotens?

(b) (1 Punkt) Was ist ein Kreis?

(c) (1 Punkt) Was bedeutet azyklisch?

(d) (1 Punkt) Was bedeutet es, eine Kante (u,v) zu relaxieren?

Lösung: a) die Anzahl der Kanten, die an dem Knoten hängen
b) ein Weg, der zurück zum Startknoten führt
c) Der Graph enthält keinen Kreis.
d) Wir prüfen, ob wir über diese Kante billiger nach v kommen, und aktualisieren gegebenenfalls dist und prev von v.